Problem 6
COM 316 Artificial Intelligence
Muhammad Abdullah, Abdul Rehman, Joseph Coombs

NLH 212 8:00pm to 9:00pm | October 13th 2026

In discrete mathematics, we can use different symbols, statements, and logical patterns in order to allow the robot to "reason" its way through different problems. However, we can expand past simple true and false statements to include what we interpret as sentences and "reasoning." For example, there are also universal and existential quantifiers that can be used to represent certain variables or predicates in certain scenarios. That way, rather than simply representing simple logic or True/False statements, the robot can learn new information about more than one object or property at a time and make inferences based on that. And, rather than having to define and write out every individual property or fact/rule about a space, we can simply just use functions to say something like `Goal(x, y)` or like `Adjacent((1,2), (2, 2), Direction))` (which could evaluate to true or false), which are much easier to write out than individually writing out every single rule.

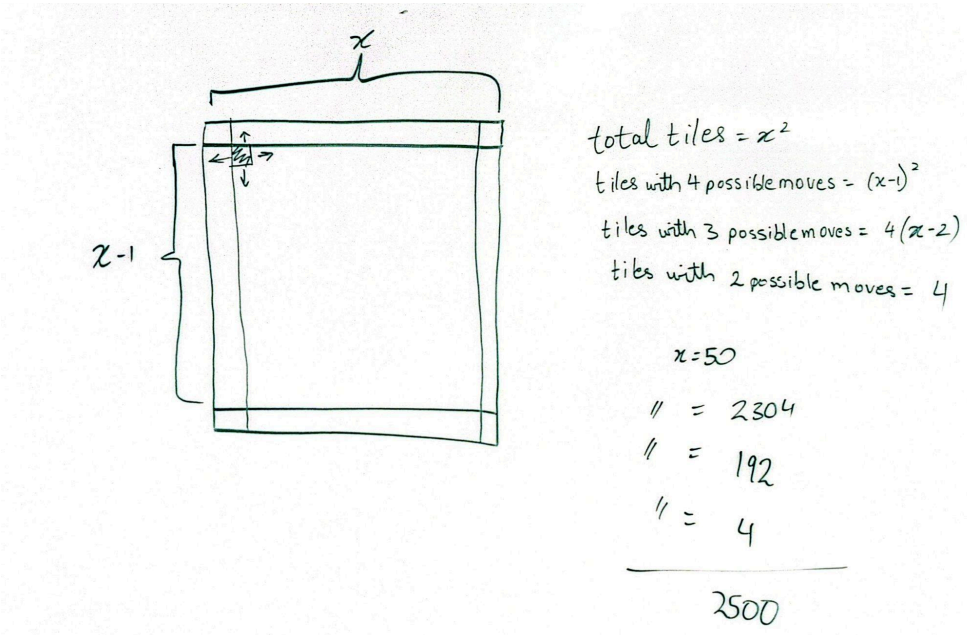
For example, instead of representing knowledge as obstacle11 or visited00, we can use variables to represent multi valued attributes and pass arguments. Since we can use variables, we can use strings or integers to represent fields that are outside the strict true/false domain. E.g. For obstacles, height can have values "low", "medium," and "high." Then, a high obstacle at x=2 and y=2 can be represented like a tuple such as:

```
(obstacle 2 2)
(height 2 2 "high")
```

Using variables would also make it easier to define the victory condition. For AI #5, we had 9 lines to check if we have attained the goal. But here, we can just see if the x and y coordinates of goal and the robot match:

```
((equal? position((visited x y)) position((goal x y))) (finished))
```

We can also use loops to define the rules instead of manually typing each and everything one by one. E.g. for a 50x50 grid, assuming there are no obstacles, there will be 48*48=2304 tiles where the robot would be allowed to move any one if any 4 directions. So, we can use a nested for loop with x and y coordinates to define these rules. The boundaries can become exceptions and we can probably use a simpler loop for them as well. By using these loops, thousands of lines of facts can be represented by a dozen lines of code.



** tiles with 4 possible moves = (x-2)^2

Our old ModusPonens2 could check every fact with one simple member call because every fact was a single self contained symbol. Now that the fact is a structure tuple like (height 2 2 “high”), checking a condition means comparing every field at once: the predicate name, the coordinates and the values. We aren’t simply just asking if the symbol is in the list or not, so reasoning over this needs a real matching function that actually walks through the facts list looking for the tuple where all the relevant positions line up rather than a membership check like in the previous program.